

รายงานงานปฏิบัติการกลุ่ม

การออกแบบและวิเคราะห์อัลกอริทึมด้วย Stack โดยใช้ภาษา Java

กลุ่มที่ 4: Expression Processor (Infix และ Postfix)

ฉบับจัดทำใหม่จากใบงานและรายงานกลุ่มเดิม

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

นิพจน์ทางคณิตศาสตร์ที่มนุษย์เขียนทั่วไปอยู่ในรูป **Infix** เช่น $3 + 4 * 2$ โดย **Operator** อยู่ระหว่าง **Operand** แต่คอมพิวเตอร์ต้องพิจารณา **Operator Precedence** และ **Parentheses** ก่อนคำนวณ จึงเหมาะที่จะใช้ **Stack** ช่วยจัดลำดับการประมวลผล.

1.2 Case Story และวัตถุประสงค์

ระบบ **Expression Processor** รับนิพจน์ **Infix** ตรวจสอบความถูกต้อง แปลง **Infix** เป็น **Postfix** และคำนวณผลลัพธ์ พร้อมเปรียบเทียบกับการประเมิน **Infix** โดยตรงด้วย **Stack** สองตัว.

- วิเคราะห์ Input, Output และข้อจำกัด
- ออกแบบ Algorithm A และ Algorithm B สำหรับปัญหาเดียวกัน
- เขียน Pseudocode และ Step-by-step
- อธิบายความถูกต้องด้วย Loop Invariant
- วิเคราะห์ Time/Space Complexity
- พัฒนาโปรแกรม Java พร้อม Test Cases และการทดลองประสิทธิภาพ

1.3 ขอบเขตงาน

- Operator: +, -, *, /
- Parentheses: ()
- Operand: จำนวนเต็มบวกและจำนวนหลัก
- รองรับช่องว่างระหว่าง Token
- ตรวจสอบวงเล็บไม่สมดุล, Operator ผิดตำแหน่ง, Division by Zero และ Empty Expression
- ใช้ Integer Division ตาม Java int / int
- ไม่บังคับรองรับทศนิยม, Unary Minus, ตัวแปร, ฟังก์ชัน และวงเล็บ [] {}

บทที่ 2 การวิเคราะห์ปัญหา

รายการ	รายละเอียด
Input	String ของ Infix เช่น "3 + 4 * 2 / (1 -

	5)"
Output	Token list, Validation result, Postfix (Algorithm A), Integer result
Constraints	ไม่รองรับทศนิยม/Unary Minus/ตัวแปร; ใช้เฉพาะ ()
Assumptions	การหารเป็น Integer Division
Edge Cases	Empty, Missing ')', Extra ')', $3 + * 4$, Division by zero

2.1 Operation ที่เกิดขึ้นบ่อย

การอ่าน Token และการ push/pop ของ Operand และ Operator เกิดขึ้นตลอดการทำงาน ส่วนการเปรียบเทียบ Precedence อาจเกิดขึ้นในช่วงที่มี Operator ความสำคัญเท่ากันหรือต่ำกว่าบน Stack.

2.2 กรณีปกติและกรณีขอบเขต

ประเภท	ตัวอย่าง	ผลที่คาดหวัง
Normal	$3 + 4 * 2$	11
Normal with parentheses	$(3 + 4) * 2$	14
Boundary	$((((2 + 3))) * 4$	20
Error	$(3 + 4$	Validation Error
Error	$3 + 4)$	Validation Error
Error	$3 + * 4$	Validation Error
Error	$10 / (5 - 5)$	ArithmeticException
Empty	""	Validation Error

บทที่ 3 การออกแบบอัลกอริทึม

3.1 Algorithm A: Infix → Postfix → Evaluation

แบ่งเป็น 2 เฟส: ใช้ Operator Stack แปลง Infix เป็น Postfix ตามหลัก Shunting-Yard แล้วใช้ Operand Stack ประเมิน Postfix.

ALGORITHM InfixToPostfix(tokens)

1. Create empty operatorStack and postfixList
2. FOR EACH token
3. IF number: append to postfixList
4. ELSE IF "(": push to operatorStack
5. ELSE IF ")": pop operators until "("; remove "("
6. ELSE operator: pop operators with precedence $\geq$ current; push current
7. END FOR
8. Pop remaining operators to postfixList
9. RETURN postfixList

ALGORITHM EvaluatePostfix(postfixTokens)

1. Create empty operandStack
2. FOR EACH token
3. IF number: push integer
4. ELSE: pop val2, pop val1, calculate val1 op val2, push result
5. RETURN the remaining value

Time Complexity: $O(n)$. Space Complexity: $O(n)$. ข้อดีคือ Postfix สามารถเก็บไว้ใช้ซ้ำได้; ข้อจำกัดคือมีการประมวลผล 2 เฟสและมี postfixList เพิ่มเติม.

3.2 Algorithm B: Direct Infix Evaluation

ประมวลผล Infix ในรอบเดียว โดยใช้ Operand Stack เก็บค่า และ Operator Stack เก็บเครื่องหมาย เมื่อพบ Operator ใหม่จะประมวลผล Operator ที่มี Precedence สูงกว่าหรือเท่ากันก่อน.

ALGORITHM EvaluateInfixDirect(tokens)

1. Create empty operandStack and operatorStack
2. FOR EACH token
3. IF number: push to operandStack

4. ELSE IF "(": push to operatorStack
5. ELSE IF ")": apply operators until "("; remove "("
6. ELSE operator: apply operators with precedence $\geq$ current; push current
7. END FOR
8. Apply all remaining operators
9. RETURN operandStack.pop()

PROCEDURE ApplyTopOperator

1. op $\leftarrow$ operatorStack.pop()
2. val2 $\leftarrow$ operandStack.pop()
3. val1 $\leftarrow$ operandStack.pop()
4. push ExecuteOperation(val1, val2, op)

Time Complexity: $O(n)$. Space Complexity: $O(n)$. ข้อดีคือ Single Pass

และไม่ต้องเก็บ Postfix; ข้อจำกัดคือ Logic รวมกันในรูปเดียวและไม่สามารถนำ Postfix ไปใช้ซ้ำได้โดยตรง.

3.3 ตัวอย่างบังคับ

Infix: $3 + 4 * 2 / (1 - 5)$

Postfix: $3\ 4\ 2\ * \ 1\ 5\ - \ / \ +$

Result: 1

Step	Token	Operator Stack	Postfix / Action
1	3	[]	3
2	+	[+]	3
3	4	[+]	3, 4
4	*	[+, *]	3, 4
5	2	[+, *]	3, 4, 2
6	/	[+, /]	Pop * $\rightarrow$ 3, 4, 2, *

7	(	[+, /, ()	unchanged
8	1	[+, /, ()	..., 1
9	-	[+, /, (, -]	..., 1
10	5	[+, /, (, -]	..., 1, 5
11	)	[+, /]	Pop -
12	End	[]	Pop /, + → final Postfix

สำหรับ **Algorithm B** เมื่ออ่านนิพจน์เดียวกัน ผลลัพธ์สุดท้ายคือ **1** เช่นเดียวกับ **Algorithm A**.

3.4 Flowcharts

Figure 3A — Algorithm A Flowchart

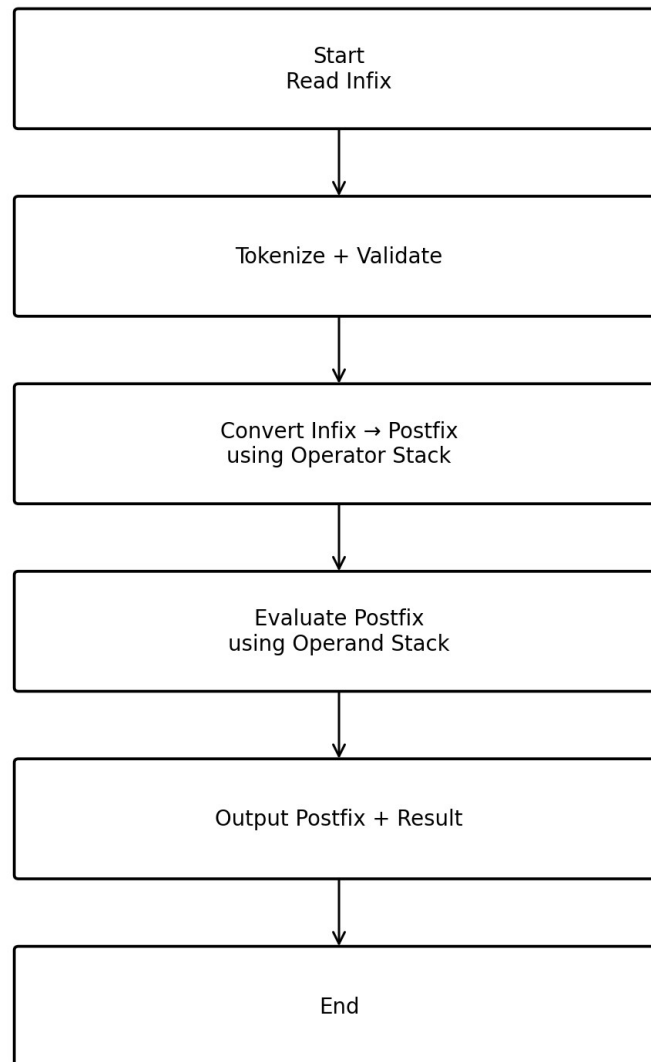
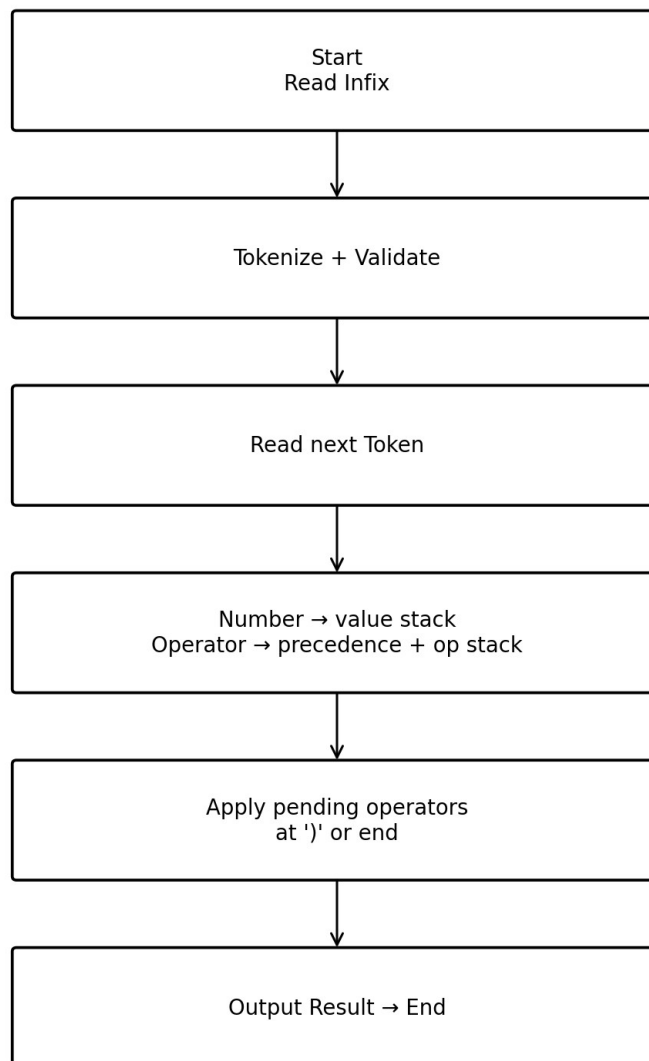


Figure 3B — Algorithm B Flowchart



บทที่ 4 การอธิบายความถูกต้อง

4.1 Precondition

Input เป็นนิพจน์ Infix ที่ผ่านการ Tokenize และ Validation แล้ว ไม่มีวงเล็บไม่สมดุล ไม่มี Operator ผิดตำแหน่ง และไม่มี Empty Expression.

4.2 Loop Invariant ของ Algorithm B

ก่อนเริ่มรอบที่ประมวลผล Token ไปแล้ว i ตัว: **Operand Stack** เก็บผลลัพธ์ของนิพจน์ย่อยที่คำนวณเสร็จจากซ้ายไปขวาโดยไม่สูญหาย และ **Operator Stack** เก็บเฉพาะ Operator/วงเล็บเปิดที่ยังรอการประมวลผล โดยลำดับ **Precedence** ภายในขอบเขตวงเล็บไม่ลดลงจากล่างขึ้นบน.

4.3 Initialization

เมื่อ $i = 0$ Stack ทั้งสองว่าง จึงเป็นจริงโดยปริยาย.

4.4 Maintenance

- **Number:** push เป็นนิพจน์ย่อยที่สมบูรณ์
- **(:** สร้าง Boundary ใหม่ใน Operator Stack
- **Operator:** ประมวลผล Operator ที่มี Precedence สูงกว่าหรือเท่ากันก่อน แล้วจึง push ตัวใหม่
- **):** ประมวลผลจนถึง '(' และลบ Boundary นั้น

4.5 Termination และ Postcondition

เมื่ออ่าน Token ครบ Operator Stack จะถูกประมวลผลจนว่าง และ Operand Stack เหลือค่าหนึ่งค่า ซึ่งเป็นผลลัพธ์ของนิพจน์ทั้งหมด. การวนซ้ำสิ้นสุดเพราะจำนวน Token มีจำกัด และแต่ละ while iteration ลดจำนวน Operator ใน Stack.

4.6 การรักษาลำดับข้อมูล

การคำนวณแบบ non-commutative รักษาลำดับซ้าย-ขวาโดยกำหนด $val2$ เป็นค่าที่ pop ครั้งแรกและ $val1$ เป็นค่าที่ pop ครั้งที่สอง แล้วคำนวณ $val1 \text{ op } val2$.

บทที่ 5 การวิเคราะห์ประสิทธิภาพ

Algorithm	Best	Average	Worst	Auxiliary Space
A: Infix→Post fix→Eval	$O(n)$	$O(n)$	$O(n)$	$O(n)$
B: Direct Infix	$O(n)$	$O(n)$	$O(n)$	$O(n)$

แม้จะมี **while** อยู่ภายใน **for** แต่การ **pop** ของแต่ละ **Token** มีจำนวนจำกัดตลอดการทำงาน จึงใช้ **Amortized Analysis** และได้ $O(n)$ ไม่ใช่ $O(n^2)$.

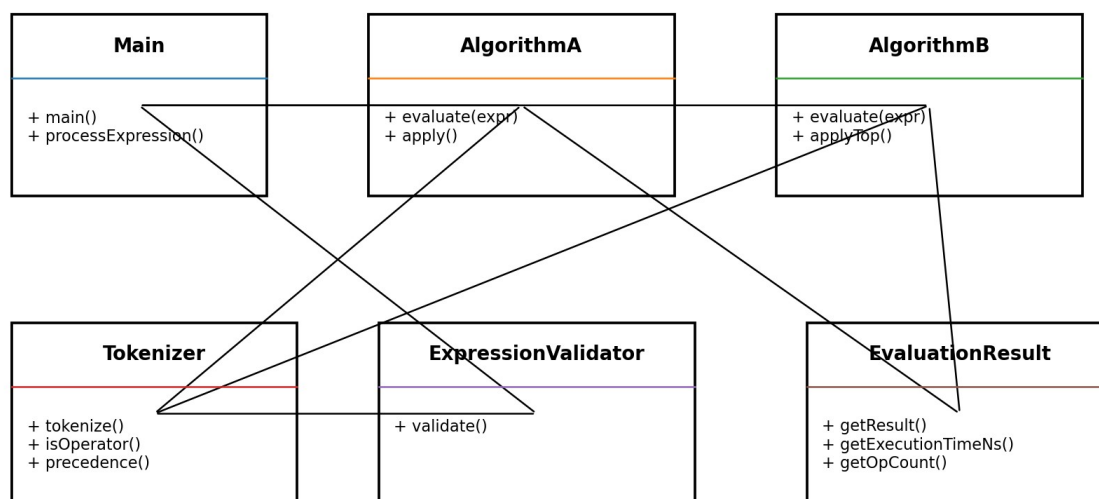
5.1 Space

Algorithm A ใช้ **Operator Stack + Postfix List + Operand Stack** รวม $O(n)$. Algorithm B ใช้ **Operand Stack + Operator Stack** รวม $O(n)$ เช่นกัน แต่ไม่มี **Postfix List** จึงมีพื้นที่ใช้งานจริงน้อยกว่าในบริบทนี้.

บทที่ 6 การพัฒนาโปรแกรม Java

Class	หน้าที่
group4.Main	เมนู รับ Input เรียก A/B และรัน Test Cases
group4.models.EvaluationResult	เก็บ Result, Postfix, Time และ Operation Counts
group4.utils.Tokenizer	แปลง String เป็น Token และกำหนด Precedence
group4.utils.ExpressionValidator	ตรวจ Empty, Parentheses และ Token Sequence
group4.algorithms.AlgorithmA	Infix → Postfix → Evaluation
group4.algorithms.AlgorithmB	Direct Infix ด้วย 2 Stacks
group4.ExpressionGenerator	สร้างนิพจน์สำหรับ Performance Test
group4.ExperimentRunner	รัน $n = 100, 1,000, 10,000, 50,000$ อย่างละ 5 รอบ

Figure 6.1 — Simplified Class Diagram



6.1 Input Validation และ Error Handling

- Invalid character → IllegalArgumentException
- Unbalanced parentheses → Validation Error
- Operator sequence ผิด → Validation Error
- Empty expression → Validation Error
- Division by zero → ArithmeticException / Execution Error
- โปรแกรมไม่ Crash และกลับสู่เมนูหลัก

บทที่ 7 ผลการทดสอบและการทดลองประสิทธิภาพ

7.1 Mandatory Test Cases

ID	Input	Expected	Actual	Status
TC01	3 + 4 * 2	Result = 11	Result = 11	PASSED
TC02	(3 + 4) * 2	Result = 14	Result = 14	PASSED
TC03	((8 + 2) * 5)	Result = 50	Result = 50	PASSED

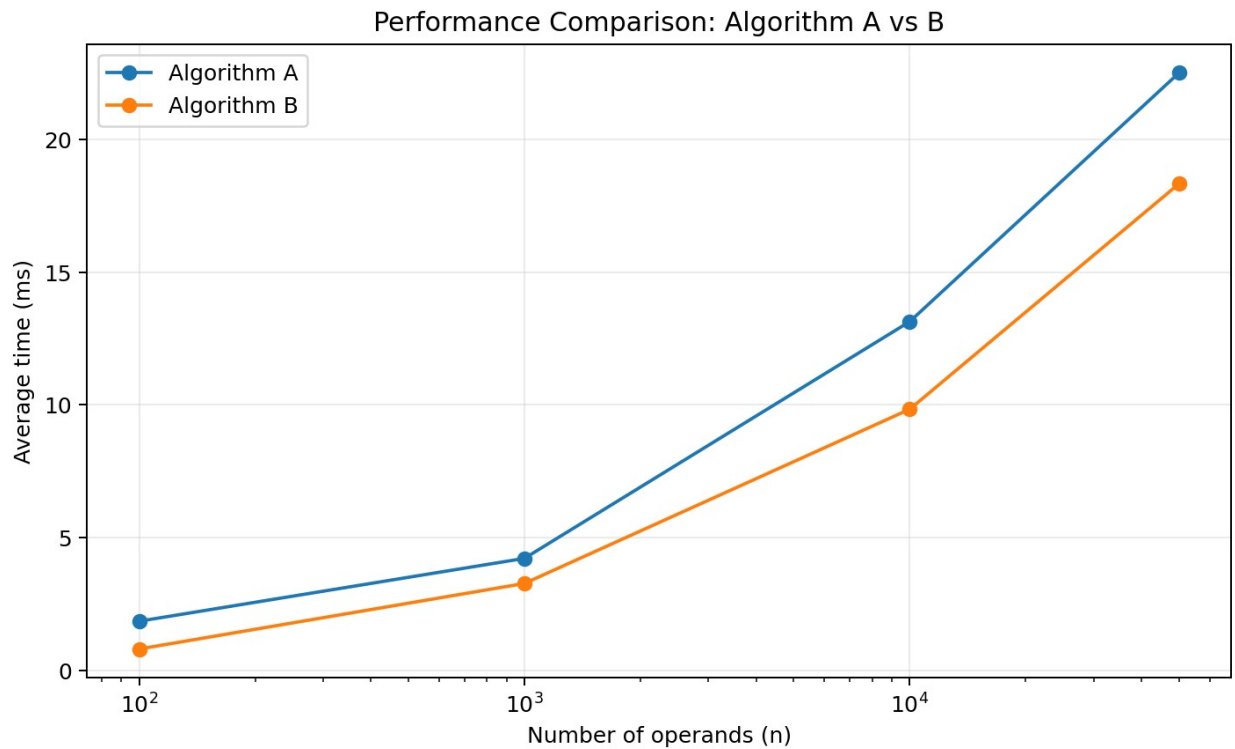
TC04	$(3 + 4$	Error: Missing ')' at end	Validation Error Caught	PASSED
TC05	$3 + 4)$	Error: Extra ')' found	Validation Error Caught	PASSED
TC06	$3 + * 4$	Error: Invalid operator sequence	Validation Error Caught	PASSED
TC07	$10 / (5 - 5)$	Error: Division by zero	Arithmetic Exception Caught	PASSED
TC08		Error: Expression is empty	Validation Error Caught	PASSED

7.2 Performance Experiment

ทดลอง $n = 100, 1,000, 10,000$ และ $50,000$ operands ขนาดละ 5 รอบ แล้วใช้ค่าเฉลี่ยของ Execution Time, Push, Pop และ Comparisons ตามข้อกำหนดในใบงาน.

n	Algorithm	Avg Time (ns)	Push	Pop	Comparisons
100	A	1,852,512	298.0	297.0	296.4
100	B	805,085	298.0	297.0	296.4
1000	A	4,219,254	2998.0	2997.0	2996.4
1000	B	3,274,884	2998.0	2997.0	2996.4
10000	A	13,138,604	29998.0	29997.0	29996.6
10000	B	9,837,999	29998.0	29997.0	29996.6

50000	A	22,509,382	149998.0	149997.0	149996.2
50000	B	18,332,269	149998.0	149997.0	149996.2



ผลการทดลองจากรายงานกลุ่มเดิมแสดงว่า **Algorithm B** เร็วกว่า **Algorithm A** ทุกขนาดที่ทดสอบ แม้ทั้งสองมี $O(n)$. ตัวอย่าง $n = 50,000$: A ประมาณ 22.5 ms และ B ประมาณ 18.3 ms.

7.3 อภิปรายผล

จำนวน **Push/Pop** เพิ่มขึ้นตามขนาดข้อมูลอย่างเป็นเชิงเส้น ซึ่งสอดคล้องกับ $O(n)$. ความแตกต่างด้านเวลามาจาก **Constant Factor**: Algorithm A มี 2 เฟสและต้องสร้าง **Postfix List** ขณะที่ Algorithm B ประมวลผลแบบ **Single Pass**.

บทที่ 8 บทที่การใช้ Generative AI

งาน	ผลลัพธ์จาก AI	วิธีตรวจสอบ	สิ่งที่กลุ่มแก้ไข
Tokenization	เสนอ <code>regex split</code>	ทดสอบ $(3+4)*2$ และตัวเลขหลักหลัก	เขียน Tokenizer แบบอ่าน <code>char</code> ทีละตัว
Big-O	เสนอ $O(n^2)$ จาก	Trace Stack +	แก้เป็น $O(n)$

	loop ซ้อน	Amortized Analysis	
Test Cases	เสนอกรณีทั่วไป	เทียบกับ 8 mandatory cases	เพิ่ม boundary/error cases
Experiment/Graph	ช่วยร่างเครื่องมือวัดผล	Compile/run และเทียบ operation counts	ใช้ helper สำหรับทดลอง โดยไม่เปลี่ยน Logic A/B

Reflection: AI ใช้เพื่อช่วยเรียนรู้และร่างแนวทางเท่านั้น ผลลัพธ์ทุกส่วนต้องถูกตรวจสอบและผู้จัดทำต้องอธิบาย Source Code ได้.

บทที่ 9 สรุปผลและการเปรียบเทียบ

9.1 ผลการเปรียบเทียบ

ประเด็น	Algorithm A	Algorithm B
แนวทาง	Infix → Postfix → Evaluate	Direct Infix
Time	$O(n)$	$O(n)$
Space	$O(n)$	$O(n)$
Postfix ใช้ซ้ำ	ได้	ไม่ได้โดยตรง
One-shot execution	ช้ากว่าในผลทดลอง	เร็วกว่าในผลทดลอง

9.2 วิธีที่เหมาะสมกว่า

สำหรับการคำนวณผลลัพธ์เพียงครั้งเดียว Algorithm B เหมาะกว่า เพราะผลทดลองแสดงเวลาต่ำกว่าและไม่ต้องสร้าง Postfix List. หากต้องเก็บ Postfix เพื่อใช้ซ้ำหรือส่งต่อให้ระบบอื่น Algorithm A มีประโยชน์มากกว่า.

9.3 ข้อจำกัดและแนวทางพัฒนา

- ยังไม่รองรับทศนิยม, Unary Minus, ตัวแปร และฟังก์ชันทางคณิตศาสตร์
- สามารถพัฒนาต่อด้วย %, ^ และ Associativity ของ ^
- เพิ่ม [] และ {}

- เพิ่มการแสดง **Trace** ของ **Stack** แบบละเอียดในโปรแกรม

9.4 สิ่งที่ได้เรียนรู้

เห็นความแตกต่างระหว่าง **Asymptotic Complexity** กับประสิทธิภาพจริงจาก **Constant Factor** และได้ฝึก **Stack, Amortized Analysis, Loop Invariant, Testing** และ **Performance Measurement**.

ภาคผนวก A: โครงสร้าง Repository สำหรับ GitHub

stack-algorithm-group04/

├─ src/

| └─ Main.java

| └─ ExpressionGenerator.java

| └─ ExperimentRunner.java

| └─ models/

| └─ algorithms/

| └─ utils/

├─ test/

| └─ test_cases.csv

├─ data/

├─ results/

| └─ experiment_results.csv

├─ diagrams/

| └─ algorithmA_flowchart.png

| └─ algorithmB_flowchart.png

| └─ class_diagram.png

| └─ performance_comparison.png

├─ presentation/

├─ report/

└─ README.md

└─ group-members.md

└─ AI_Usage_Log.md

ตามใบงาน สมาชิกทุกคนต้องมี **Commit** ใน **Repository** และต้องสามารถอธิบาย **Source Code** ของกลุ่มได้. ตัวอย่าง **Commit: Implement expression processor; Add mandatory test cases; Add performance experiment; Update complexity analysis.**

ภาคผนวก B: Source Code

src/Main.java

```
package group4;

import group4.algorithms.AlgorithmA;
import group4.algorithms.AlgorithmB;
import group4.models.EvaluationResult;
import group4.utils.ExpressionValidator;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.println("=====");
        System.out.println(" Group 4: Expression Processor (Infix & Postfix)");
        System.out.println("=====");

        while (true) {

            System.out.println("\nSelect Menu:");

            System.out.println("1. Test Custom Expression");
```

```

System.out.println("2. Run Mandatory Assignment Test Cases (1-8)");

System.out.println("3. Exit");

System.out.print("Enter choice (1-3): ");

String choice = scanner.nextLine().trim();


if (choice.equals("3")) break;

if (choice.equals("1")) {

    System.out.print("Enter Infix Expression: ");

    processExpression(scanner.nextLine());

} else if (choice.equals("2")) {

    runMandatoryTestCases();

} else {

    System.out.println("Invalid choice. Please enter 1, 2, or 3.");

}

}

scanner.close();

}

```

```

private static void processExpression(String expr) {

    System.out.println("\n--- Processing: '\"' + expr + "\"' ---");

    try {

        ExpressionValidator.validate(expr);

        EvaluationResult a = AlgorithmA.evaluate(expr);

        EvaluationResult b = AlgorithmB.evaluate(expr);


        System.out.println("[Algorithm A - Infix to Postfix]");

        System.out.println("Postfix : " + a.getPostfixExpression());
    }
}

```

```

        System.out.println("Result : " + a.getResult());

        System.out.println("Time   : " + a.getExecutionTimeNs() + " ns");

        System.out.println("Ops    : " + a.getOpCount());

        System.out.println("\n[Algorithm B - Direct Infix]");

        System.out.println("Result : " + b.getResult());

        System.out.println("Time   : " + b.getExecutionTimeNs() + " ns");

        System.out.println("Ops    : " + b.getOpCount());

    } catch (IllegalArgumentException e) {

        System.out.println("Validation Error: " + e.getMessage());

    } catch (ArithmeticException e) {

        System.out.println("Execution Error: " + e.getMessage());

    }

}
}

```

```

private static void runMandatoryTestCases() {

    String[] testCases = {

        "3 + 4 * 2",

        "(3 + 4) * 2",

        "((8 + 2) * 5)",

        "(3 + 4",

        "3 + 4)",

        "3 + * 4",

        "10 / (5 - 5)",

        ""

    };

    for (int i = 0; i < testCases.length; i++) {

        System.out.println("\n=== Mandatory Test Case " + (i + 1) + " ===");
    }
}

```

```

        processExpression(testCases[i]);
    }
}
}

```

src/utls/Tokenizer.java

```

package group4.utls;

import java.util.ArrayList;
import java.util.List;

public class Tokenizer {

    public static List<String> tokenize(String expr) {

        List<String> tokens = new ArrayList<>();

        StringBuilder number = new StringBuilder();

        for (int i = 0; i < expr.length(); i++) {

            char c = expr.charAt(i);

            if (Character.isWhitespace(c)) {

                if (number.length() > 0) {

                    tokens.add(number.toString());

                    number.setLength(0);

                }

            } else if (Character.isDigit(c)) {

                number.append(c);

            } else if (c == '(' || c == ')' || isOperatorChar(c)) {

                if (number.length() > 0) {

                    tokens.add(number.toString());

                    number.setLength(0);

                }

                tokens.add(c + "");

            }
        }

        if (number.length() > 0) {

            tokens.add(number.toString());

        }

        return tokens;

    }

}

```

```

        tokens.add(number.toString());

        number.setLength(0);
    }

    tokens.add(String.valueOf(c));
} else {

    throw new IllegalArgumentException(

        "Invalid character found in expression: " + c

    );
}
}

if (number.length() > 0) tokens.add(number.toString());

return tokens;
}

```

```

public static boolean isOperator(String token) {

    return token.equals("+") || token.equals("-")

        || token.equals("*") || token.equals("/");
}

```

```

private static boolean isOperatorChar(char c) {

    return c == '+' || c == '-' || c == '*' || c == '/';
}

```

```

public static int precedence(String op) {

    switch (op) {

        case "+": case "-": return 1;

```

```

        case "*": case "/": return 2;

        default: return 0;
    }
}
}

```

src/utls/ExpressionValidator.java

```

package group4.utls;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.List;

public class ExpressionValidator {

    public static void validate(String expr) {

        if (expr == null || expr.trim().isEmpty()) {

            throw new IllegalArgumentException("Expression is empty.");

        }

        List<String> tokens = Tokenizer.tokenize(expr);

        if (tokens.isEmpty()) {

            throw new IllegalArgumentException("Expression contains no valid tokens.");

        }

        Deque<Character> stack = new ArrayDeque<>();

        for (char c : expr.toCharArray()) {

            if (c == '(') stack.push(c);

            else if (c == ')') {

```

```

        if (stack.isEmpty()) {
            throw new IllegalArgumentException("Unbalanced parentheses: Extra ')' found.");
        }
        stack.pop();
    }
}

if (!stack.isEmpty()) {
    throw new IllegalArgumentException("Unbalanced parentheses: Missing ')' at end.");
}

boolean expectOperand = true;
for (String token : tokens) {
    if (Tokenizer.isOperator(token)) {
        if (expectOperand) {
            throw new IllegalArgumentException("Invalid operator sequence near '" + token + "'");
        }
        expectOperand = true;
    } else if (token.equals("(")) {
        if (!expectOperand) {
            throw new IllegalArgumentException("Missing operator before '('");
        }
        expectOperand = true;
    } else if (token.equals(")")) {
        if (expectOperand) {
            throw new IllegalArgumentException("Invalid ')' placement");
        }
        expectOperand = false;
    } else {

```

```

        if (!expectOperand) {

            throw new IllegalArgumentException("Missing operator between operands");

        }

        expectOperand = false;

    }

}

if (expectOperand) {

    throw new IllegalArgumentException("Expression ends with an operator or empty parentheses.");

}

}

}

```

src/models/EvaluationResult.java

```

package group4.models;

public class EvaluationResult {

    private final String postfixExpression;

    private final int result;

    private final long executionTimeNs;

    private final int pushCount;

    private final int popCount;

    private final int comparisonCount;

    public EvaluationResult(String postfixExpression, int result, long executionTimeNs,

        int pushCount, int popCount, int comparisonCount) {

        this.postfixExpression = postfixExpression;

        this.result = result;
    }
}

```

```
        this.executionTimeNs = executionTimeNs;

        this.pushCount = pushCount;

        this.popCount = popCount;

        this.comparisonCount = comparisonCount;
    }
}
```

```
public String getPostfixExpression() { return postfixExpression; }

public int getResult() { return result; }

public long getExecutionTimeNs() { return executionTimeNs; }

public int getPushCount() { return pushCount; }

public int getPopCount() { return popCount; }

public int getComparisonCount() { return comparisonCount; }
```

```
public String getOpCount() {

    return "Push=" + pushCount + ", Pop=" + popCount + ", Comp=" + comparisonCount;

}

}
```

[src/algorithms/AlgorithmA.java](#)

```
package group4.algorithms;

import group4.models.EvaluationResult;

import group4.utils.Tokenizer;

import java.util.ArrayDeque;

import java.util.ArrayList;

import java.util.Deque;

import java.util.List;
```

```

public class AlgorithmA {

    public static EvaluationResult evaluate(String expr) {

        long start = System.nanoTime();

        int push = 0, pop = 0, comp = 0;


        List<String> tokens = Tokenizer.tokenize(expr);

        List<String> postfix = new ArrayList<>();

        Deque<String> opStack = new ArrayDeque<>();


        for (String token : tokens) {

            comp++;

            if (Character.isDigit(token.charAt(0))) {

                postfix.add(token);

            } else if (token.equals("(")) {

                opStack.push(token); push++;

            } else if (token.equals(")")) {

                while (!opStack.isEmpty() && !opStack.peek().equals("(")) {

                    postfix.add(opStack.pop()); pop++;

                }

                if (opStack.isEmpty()) throw new ArithmeticException("Invalid parentheses.");

                opStack.pop(); pop++;

            } else {

                while (!opStack.isEmpty()

                    && Tokenizer.precedence(opStack.peek()) >= Tokenizer.precedence(token)) {

                    comp++;

                    postfix.add(opStack.pop()); pop++;

                }

                opStack.push(token); push++;
            }
        }
    }
}

```

```

    }
}

while (!opStack.isEmpty()) {
    String op = opStack.pop();
    if (op.equals("(")) throw new ArithmeticException("Invalid parentheses.");
    postfix.add(op); pop++;
}

Deque<Integer> values = new ArrayDeque<>();
for (String token : postfix) {
    if (Character.isDigit(token.charAt(0))) {
        values.push(Integer.parseInt(token)); push++;
    } else {
        if (values.size() < 2) throw new ArithmeticException("Invalid postfix structure.");
        int b = values.pop(), a = values.pop(); pop -= 2;
        if (token.equals("/") && b == 0) throw new ArithmeticException("Division by zero.");
        values.push(apply(a, b, token)); push++;
    }
}

long end = System.nanoTime();
return new EvaluationResult(String.join(" ", postfix), values.pop(),
    end - start, push, pop, comp);
}

```

```

private static int apply(int a, int b, String op) {

```

```

switch (op) {

    case "+": return a + b;

    case "-": return a - b;

    case "*": return a * b;

    case "/": return a / b;

    default: throw new IllegalArgumentException("Unsupported operator: " + op);

}

}

}

```

src/algorithms/AlgorithmB.java

```

package group4.algorithms;


import group4.models.EvaluationResult;

import group4.utils.Tokenizer;

import java.util.ArrayDeque;

import java.util.Deque;

import java.util.List;


public class AlgorithmB {

    public static EvaluationResult evaluate(String expr) {

        long start = System.nanoTime();

        int push = 0, pop = 0, comp = 0;


        List<String> tokens = Tokenizer.tokenize(expr);

        Deque<Integer> values = new ArrayDeque<>();

        Deque<String> ops = new ArrayDeque<>();
    }
}

```

```

for (String token : tokens) {

    comp++;

    if (Character.isDigit(token.charAt(0))) {

        values.push(Integer.parseInt(token)); push++;

    } else if (token.equals("(")) {

        ops.push(token); push++;

    } else if (token.equals("")) {

        while (!ops.isEmpty() && !ops.peek().equals("(")) {

            int[] c = applyTop(values, ops);

            push += c[0]; pop += c[1]; comp += c[2];

        }

        if (ops.isEmpty()) throw new ArithmeticException("Invalid parentheses.");

        ops.pop(); pop++;

    } else {

        while (!ops.isEmpty()

            && Tokenizer.precedence(ops.peek()) >= Tokenizer.precedence(token)) {

            comp++;

            int[] c = applyTop(values, ops);

            push += c[0]; pop += c[1]; comp += c[2];

        }

        ops.push(token); push++;

    }

}

while (!ops.isEmpty()) {

    int[] c = applyTop(values, ops);

    push += c[0]; pop += c[1]; comp += c[2];

}

```

```

        long end = System.nanoTime();

        return new EvaluationResult("N/A (Direct Infix)", values.pop(),

            end - start, push, pop, comp);
    }

```

```

private static int[] applyTop(Deque<Integer> values, Deque<String> ops) {

    String op = ops.pop();

    int b = values.pop();

    int a = values.pop();

    if (op.equals("/") && b == 0) throw new ArithmeticException("Division by zero.");

    int r;

    switch (op) {

        case "+": r = a + b; break;

        case "-": r = a - b; break;

        case "*": r = a * b; break;

        case "/": r = a / b; break;

        default: throw new IllegalArgumentException("Unsupported operator: " + op);

    }

    values.push(r);

    return new int[]{1, 3, 0};

}

```

src/ExpressionGenerator.java

```

package group4;

```

```

import java.util.Random;

```

```

public class ExpressionGenerator {

    private static final char[] OPS = {'+', '-', '*', '/'};


    public static String generate(int numberOfOperands, long seed) {

        Random random = new Random(seed);

        StringBuilder sb = new StringBuilder();


        for (int i = 0; i < numberOfOperands; i++) {

            int value = 1 + random.nextInt(9);

            sb.append(value);

            if (i < numberOfOperands - 1) {

                sb.append(' ').append(OPS[random.nextInt(OPS.length)]).append(' ');

            }

        }

        return sb.toString();

    }

}

```

src/ExperimentRunner.java

```

package group4;


import group4.algorithms.AlgorithmA;

import group4.algorithms.AlgorithmB;

import group4.models.EvaluationResult;

import java.io.File;

import java.io.FileWriter;

import java.io.IOException;

```

```

import java.io.PrintWriter;

public class ExperimentRunner {

    private static final int[] SIZES = {100, 1000, 10000, 50000};

    private static final int ROUNDS = 5;

    public static void main(String[] args) throws IOException {

        new File("results").mkdirs();

        run("results/experiment_results.csv");

    }

    public static void run(String outputCsvPath) throws IOException {

        try (PrintWriter csv = new PrintWriter(new FileWriter(outputCsvPath))) {

            csv.println("n,algorithm,avg_time_ns,avg_push,avg_pop,avg_comparisons");

            for (int n : SIZES) {

                long[] ta = new long[ROUNDS], pa = new long[ROUNDS], qa = new long[ROUNDS], ca = new
long[ROUNDS];

                long[] tb = new long[ROUNDS], pb = new long[ROUNDS], qb = new long[ROUNDS], cb = new
long[ROUNDS];

                for (int r = 0; r < ROUNDS; r++) {

                    String expr = ExpressionGenerator.generate(n, 1000L * n + r);

                    EvaluationResult a = AlgorithmA.evaluate(expr);

                    ta[r] = a.getExecutionTimeNs();

                    pa[r] = a.getPushCount();

```

```

        qa[r] = a.getPopCount();

        ca[r] = a.getComparisonCount();

        EvaluationResult b = AlgorithmB.evaluate(expr);

        tb[r] = b.getExecutionTimeNs();

        pb[r] = b.getPushCount();

        qb[r] = b.getPopCount();

        cb[r] = b.getComparisonCount();

    }

    write(csv, n, "A", avg(ta), avg(pa), avg(qa), avg(ca));

    write(csv, n, "B", avg(tb), avg(pb), avg(qb), avg(cb));

}

System.out.println("Saved results to: " + outputCsvPath);

}

private static void write(PrintWriter csv, int n, String alg,

        double t, double p, double q, double c) {

    csv.printf("%d,%s,%0f,%0.1f,%0.1f,%0.1f%n", n, alg, t, p, q, c);

}

private static double avg(long[] v) {

    long sum = 0;

    for (long x : v) sum += x;

    return (double) sum / v.length;

}

```

}